



# VISUAL BASIC/ C# PROGRAMMING (330)

NATIONAL – 2019

**Knowledge Test:**

Certification Exam: MTA Software Development Fundamentals \_\_\_\_\_ (100 points)

**Production Portion:**

Program 1: Player Inventory \_\_\_\_\_ (550 points)

***TOTAL POINTS*** \_\_\_\_\_ ***(650 points)***

**Graders: Please double check and verify all scores and answer keys!**

Property of Business Professionals of America.  
May be reproduced only for use in the Business Professionals of America  
*Workplace Skills Assessment Program* competition.



## Player Inventory

In this exercise, you will create a Visual Basic or C# Windows Form Application that allows the user to view inventory items and buy from / sell to a shop. Auto-quest allows the player to acquire more gold and the shop can be refreshed to view additional items and replenish gold.

Requirements:

1. You must create a C#/VB Windows Form Application named CS\_330\_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS\_330\_01\_2345\_6789. If you are using VB, use VB instead of CS in the application name.
2. Your name and contestant number must appear as a comment at the top of the main source code file.
3. Application on Exit button press must ask for confirmation to close, with the Title of the confirmation box being “Confirm” and the text being “Are you sure you wish to exit?”
  - a. Exit button must be named “exitButton”
4. The main form must not have minimize or maximize, or close buttons in the Form Handle.
5. The main form must have text “Player Inventory” for its title.
6. A “Item” class must be created containing the following:
  - a. Data members string name, double weight, double value, and string image
  - b. Constructor for assigning data members and accessors
  - c. A virtual function named GetInformationString that returns a string to use for the “itemInfoLabel”.
  - d. Hint: In order for Serialization calls to work with “Item” class objects, add the following XmlInclude tags to the Item class definition:

```
[XmlInclude(typeof(Weapon))]  
[XmlInclude(typeof(Armor))]  
16 references  
public class Item
```

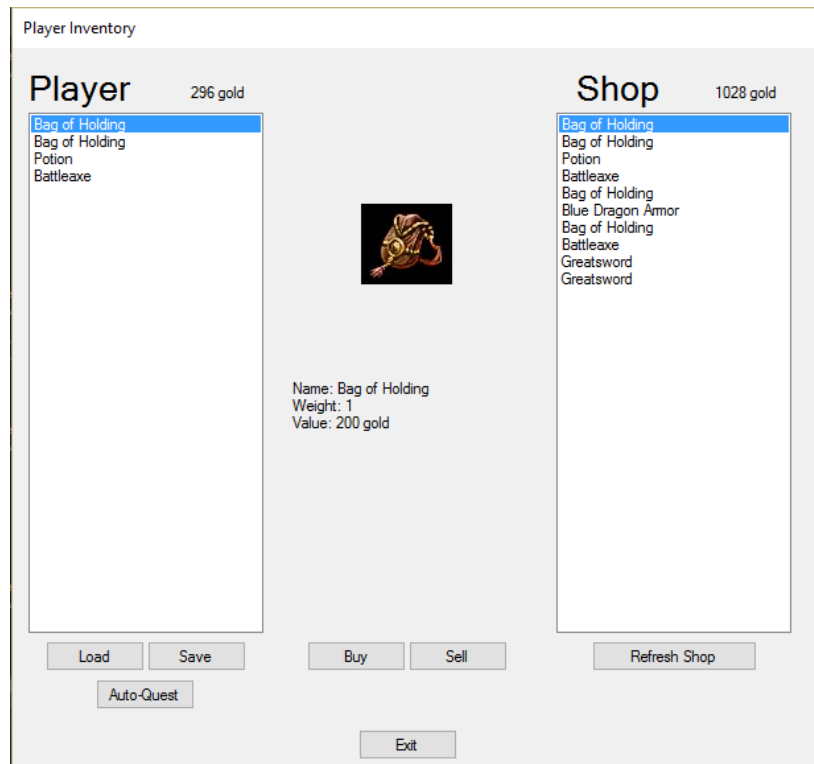
7. A “Weapon” class must be created, inherited from the “Item” class.
  - a. Data member string baseDamage is added.
  - b. Constructor for assigning data members and accessors.
  - c. GetInformationString override that includes the base class information, plus the baseDamage.
8. An “Armor” class must be created, inherited from the “Item” class.
  - a. Data member string baseArmor is added.
  - b. Constructor for assigning data members and accessors.
  - c. GetInformationString override that includes the base class information, plus the baseArmor.
9. A “Character” class must be created containing the following:
  - a. Data members double gold and List<Item> items.
  - b. Accessors, BuyItem, and SellItem member functions.



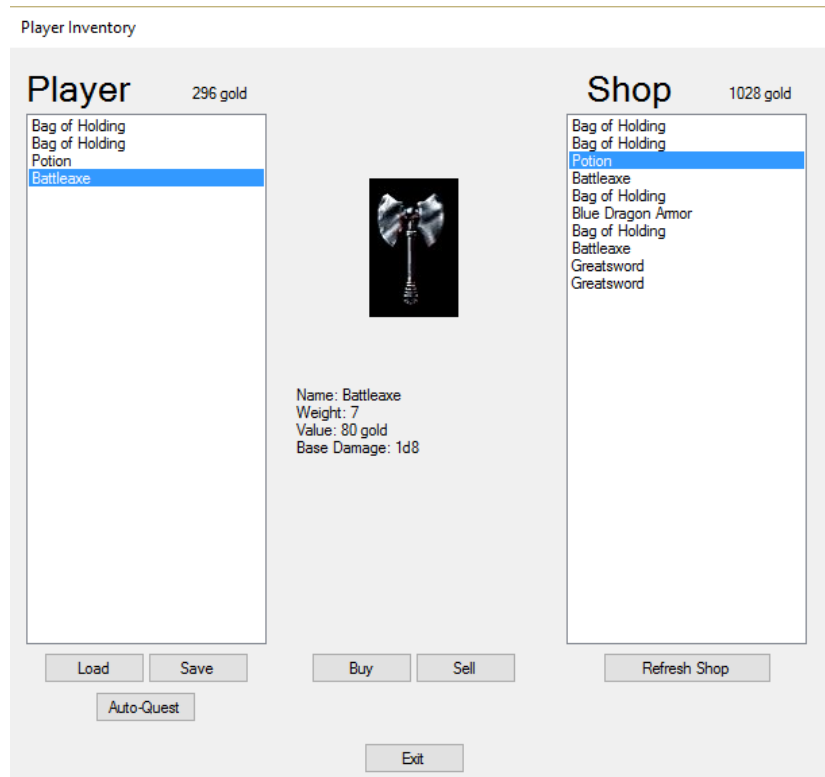
10. On startup, “all\_items.xml” must be loaded into global “List<Item> allItems” using the provided “Serialization.cs”.
11. On startup, a “Character player” must be populated as a character with four random items from “allItems”, 200 base gold, and 0-200 additional gold (selected randomly).
  - a. Player items must be shown in a list view on the left side of the screen.
    - i. The first item must be selected on startup.
  - b. “Player” must be displayed above and aligned to the left of the list box.
  - c. “### gold” must be displayed above and aligned to the right of the list box.
12. On startup, a “Character shop” must be populated as a character with ten random items from “allItems”, 1000 base gold, and 0-200 additional gold (selected randomly).
  - a. Shop items must be shown in a list view on the right side of the screen.
    - i. The first item must be selected on startup.
  - b. “Shop” must be displayed above and aligned to the right of the list box.
  - c. “### gold” must be displayed above and aligned to the right of the list box.
  - d. The main form must have a “Refresh Shop” button that repopulates “Character shop” with new random items and gold.
13. The main form must have a “Load” button that allows the user to load a player from file using an OpenFileDialog.
  - a. Attempts to load an invalid file must report “Invalid player file selected!” using a MessageBox.
14. The main form must have a “Save” button that allows the user to save a player to file using a SaveFileDialog.
15. The main form must have an “Auto-Quest” button that will give the player a base 20 gold plus an additional 0-20 gold when clicked.
16. The main form must include the image and description of the last selected item from either the player or shop list box.
17. The main form must have a “Buy” button that allows the player to purchase the selected shop item.
  - a. If player does not have enough gold, an “Insufficient gold!” MessageDialog must be displayed and the transaction cancelled.
  - b. If player does have enough gold, the item is added to the player and removed from the shop. The value of the item is removed from player gold and added to the shop gold.
18. The main form must have a “Sell” button that allows the player to sell the selected player item.
  - a. If the shop does not have enough gold, an “Insufficient gold!” MessageDialog must be displayed and the transaction cancelled.
  - b. If the shop does have enough gold, the item is added to the shop and removed from the player. The value of the item is removed from the shop gold and added to the player gold.



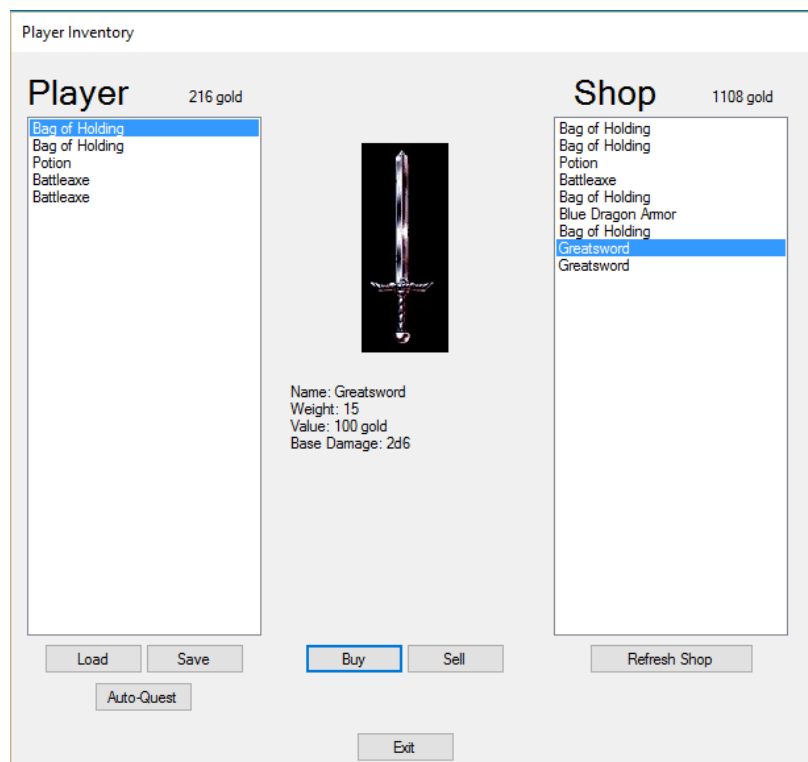
**Sample Pictures:**



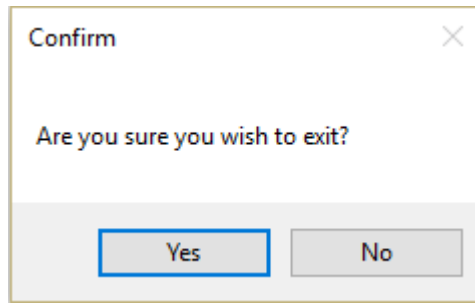
**Figure 1:** Main screen after form loads.



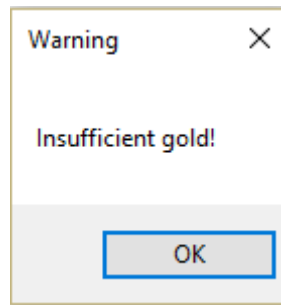
**Figure 2:** Item information in the center is the last selected item from either player or shop.



**Figure 3:** Selecting a Battleaxe and clicking “Buy” results in the Battleaxe removed from shop and added to the player.



**Figure 4:** The exit confirmation window. This should be opened when Exit is pressed. If “Yes” is clicked, the application should close, otherwise return to the main form.



**Figure 4:** Insufficient gold! MessageBox.

You will have 90 minutes to complete your work.

Your name and/or school name should NOT appear on any work you submit for grading.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

#### Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).



Your application will be graded on the following criteria:

### **Solution and Project**

The project is present on the flash drive \_\_\_\_\_ 10 pts  
The project is named according to the naming conventions \_\_\_\_\_ 10 pts

### **Program Execution**

Code copied to USB drive and program runs from USB \_\_\_\_\_ 20 pts

*If the program does not execute, then the remaining items in the program executive section receive a score of zero.*

The program contains an Exit button that asks for user confirmation \_\_\_\_\_ 10 pts  
The main form does not have handle controls (minimize, maximize, close) \_\_\_\_\_ 10 pts  
On startup, player has four items and 200-400 gold \_\_\_\_\_ 25 pts  
On startup, shop has ten items and 1000-1200 gold \_\_\_\_\_ 25 pts  
Clicking the Refresh Shop button randomly selects ten new items and  
a new amount of gold between 1000-1200 gold \_\_\_\_\_ 20 pts  
Selecting an item from the player list shows the image and description \_\_\_\_\_ 20 pts  
Selecting an item from the shop list shows the image and description \_\_\_\_\_ 20 pts  
Clicking the Buy button:  
    Player gold is correctly checked \_\_\_\_\_ 20 pts  
    If sufficient gold, selected item is transferred from shop to player \_\_\_\_\_ 20 pts  
    If sufficient gold, player and shop gold are updated correctly \_\_\_\_\_ 20 pts  
Clicking the Sell button:  
    Shop gold is correctly checked \_\_\_\_\_ 20 pts  
    If sufficient gold, selected item is transferred from player to shop \_\_\_\_\_ 20 pts  
    If sufficient gold, player and shop gold are updated correctly \_\_\_\_\_ 20 pts  
Clicking the Load button opens dialog loads selected player file \_\_\_\_\_ 25 pts  
Clicking the Save button opens dialog saves active player to file \_\_\_\_\_ 25 pts  
Clicking the Auto-Quest button adds 20-40 gold to the player \_\_\_\_\_ 10 pts

### **Source Code Review**

Code is commented at the top, for each function, and as needed \_\_\_\_\_ 10 pts  
Code uses reasonable and consistent variable naming conventions \_\_\_\_\_ 10 pts  
An “Item” class is implemented with appropriate members \_\_\_\_\_ 10 pts  
A “Weapon” class and an “Armor” class are implemented that  
    inherit from “Item” with override for GetInformationString \_\_\_\_\_ 10 pts  
A “Character” class is implemented with appropriate members \_\_\_\_\_ 10 pts  
Serialization file is imported into project \_\_\_\_\_ 10 pts  
Source to create random character is present \_\_\_\_\_ 10 pts  
Source to generate random shop items is present \_\_\_\_\_ 10 pts  
Source to update Player GUI is present \_\_\_\_\_ 20 pts



|                                                                       |              |
|-----------------------------------------------------------------------|--------------|
| Source to update Shop GUI is present                                  | _____ 10 pts |
| Source to control user interaction (shop/player selection) is present | _____ 20 pts |
| Source to control user interaction (buying/selling items) is present  | _____ 20 pts |
| Source to generate random gold value is present                       | _____ 10 pts |
| Source to load player information (from xml) is present               | _____ 20 pts |
| Source to save player information (to xml) is present                 | _____ 20 pts |

Total Points: 550 pts





C# SOURCE:

```
Form1.cs
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Xml;
using System.Xml.Serialization;

namespace player_inventory
{
    public partial class Form1 : Form
    {
        List<Item> allItems = new List<Item>();

        Character player = new Character();
        Character shop = new Character();

        public Form1()
        {
            InitializeComponent();

            try
            {
                // Attempt to load all items
                allItems = Serialization.Deserialize<List<Item>>("all_items.xml");
            }
            catch
            {
                // Development Only - generate items file
                GenerateAllItemsFile();
            }

            NewPlayer();
            NewShop();
        }

        // Development Only - generate items file
        private void GenerateAllItemsFile()
        {
            allItems.Add(new Weapon("Greatsword", 15.0, 100, "../images/greatsword.png", "2d6"));
            allItems.Add(new Item("Potion", 0.5, 10, "../images/potion.png"));
            allItems.Add(new Item("Bag of Holding", 1.0, 200, "../images/bag_of_holding.png"));
            allItems.Add(new Weapon("Battleaxe", 7.0, 80, "../images/battleaxe.png", "1d8"));
            allItems.Add(new Armor("Blue Dragon Armor", 40.0, 300, "../images/blue_dragon_armor.png", "8"));

            Serialization.Serialize(allItems, "all_items.xml");
        }

        // Create new random player and update GUI
        private void NewPlayer()
```



```

    {
        player = CreateRandomCharacter(4, 200, 200);
        UpdatePlayerGui();
    }

// Create new random shop and update GUI
private void NewShop()
{
    shop = CreateRandomCharacter(10, 1000, 200);
    UpdateVendorGui();
}

// Create random character
// numItems:    Number of random items to give the new character from overall item list
// baseGold:    Initial amount of character gold
// randomExtraGold: param value * random (0-1) added to baseGold to determine character gold
private Character CreateRandomCharacter(int numItems, double baseGold, double randomExtraGold)
{
    Character temp = new Character();

    Random rng = new Random();
    for (int i = 0; i < numItems; i++)
    {
        int randomIndex = (int)Math.Round(rng.NextDouble() * (allItems.Count - 1));
        temp.items.Add(allItems[randomIndex]);
    }
    temp.gold = rng.NextDouble() * randomExtraGold + baseGold;

    return temp;
}

private void UpdateVendorGui()
{
    label_vendorGold.Text = Math.Round(shop.gold).ToString() + " gold"; // format gold display
    shopList.DataSource = null; // DataSource must be changed for updates to take effect
    shopList.DataSource = shop.items;
    shopList.DisplayMember = "name";

    // Set SelectedIndex to 0 if nothing is currently selected and there are items available
    if ((shopList.SelectedIndex == -1) && (shop.items.Count != 0))
    {
        shopList.SelectedIndex = 0;
    }
}

private void UpdatePlayerGui()
{
    label_playerGold.Text = Math.Round(player.gold).ToString() + " gold"; // format gold display
    playerList.DataSource = null; // DataSource must be changed for updates to take effect
    playerList.DataSource = player.items;
    playerList.DisplayMember = "name";

    // Set SelectedIndex to 0 if nothing is currently selected and there are items available
    if ((playerList.SelectedIndex == -1) && (player.items.Count != 0))
    {
        playerList.SelectedIndex = 0;
    }
}

```



```

    }

    // Update item info image and description
    private void SetItemInfo(Item item)
    {
        itemInfoPictureBox.Image = Image.FromFile(item.image);
        itemInfoLabel.Text = item.GetInformationString();
    }

#region GUI_Events-Lists
    private void shopList_SelectedIndexChanged(object sender, EventArgs e)
    {
        // set item info if item is selected
        if (shopList.SelectedItem != null)
        {
            SetItemInfo((Item)shopList.SelectedItem);
        }
    }

    private void playerList_SelectedIndexChanged(object sender, EventArgs e)
    {
        // set item info if item is selected
        if (playerList.SelectedItem != null)
        {
            SetItemInfo((Item)playerList.SelectedItem);
        }
    }
#endregion

#region GUI_Events-Buttons
    private void refreshShopButton_Click(object sender, EventArgs e)
    {
        NewShop();
    }

    private void exitButton_Click(object sender, EventArgs e)
    {
        DialogResult r1 = MessageBox.Show("Are you sure you wish to exit?", "Confirm",
        MessageBoxButtons.YesNo);
        if (r1 == DialogResult.Yes)
        {
            Application.Exit();
        }
    }

    private void sellButton_Click(object sender, EventArgs e)
    {
        // Check that a item is selected in the player list
        if (playerList.SelectedIndex != -1)
        {
            // If shop is successful in buying player's item...
            if (shop.BuyItem(player.items[playerList.SelectedIndex]))
            {
                // Player sells item
                player.SellItem(playerList.SelectedIndex);
            }
        }
    }

```



```

    }
    UpdatePlayerGui();
    UpdateVendorGui();
}

}

```

```

private void buyButton_Click(object sender, EventArgs e)
{
    // Check that a item is selected in the shop list
    if (shopList.SelectedIndex != -1)
    {
        // If player is successful in buying shop's item...
        if (player.BuyItem(shop.items[shopList.SelectedIndex]))
        {
            // Shops sells item
            shop.SellItem(shopList.SelectedIndex);
        }
        UpdatePlayerGui();
        UpdateVendorGui();
    }
}

```

```

// Generate random gold value between 20 and 40 to add to player
private void autoQuestButton_Click(object sender, EventArgs e)
{
    Random rng = new Random();
    player.gold += rng.NextDouble() * 20 + 20;
    UpdatePlayerGui();
}

```

```

// Load player
private void loadButton_Click(object sender, EventArgs e)
{
    OpenFileDialog openFileDialog1 = new OpenFileDialog();
    openFileDialog1.InitialDirectory = Environment.CurrentDirectory;
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        try
        {
            player = Serialization.Deserialize<Character>(openFileDialog1.FileName);
        }
        catch
        {
            MessageBox.Show("Invalid player file selected!", "Warning");
        }
    }
    UpdatePlayerGui();
}

```

```

// Save player
private void saveButton_Click(object sender, EventArgs e)
{
    SaveFileDialog saveFileDialog1 = new SaveFileDialog();

```



## ANSWER KEY

Page 13 of 21

```

        saveFileDialog1.InitialDirectory = Environment.CurrentDirectory;
        if (saveFileDialog1.ShowDialog() == DialogResult.OK)
        {
            try
            {
                Serialization.Serialize(player, saveFileDialog1.FileName);
            }
            catch
            {
                MessageBox.Show("Invalid player file selected!", "Warning");
            }
        }
    }
}
#endregion
}

```

```

#region CharacterClass
public class Character
{
    public double gold { get; set; }
    public List<Item> items { get; set; }

```

```

    public Character()
    {
        items = new List<Item>();
        gold = 0;
    }

```

```

    public bool BuyItem(Item item)
    {
        bool status = true;
        if (gold < item.value)
        {
            MessageBox.Show("Insufficient gold!", "Warning");
            status = false;
        }
        else
        {
            gold -= item.value;
            items.Add(item);
        }
        return status;
    }

```

```

    public bool SellItem(int itemIndex)
    {
        gold += items[itemIndex].value;
        items.RemoveAt(itemIndex);
        return true;
    }
}

```

#endregion

```

#region ItemClass
[XmlInclude(typeof(Weapon))] // required for serialization
[XmlInclude(typeof(Armor))] // required for serialization

```



```

public class Item
{
    public string name { get; set; }
    public double weight { get; set; }
    public double value { get; set; }
    public string image { get; set; }

    public Item(){}

    public Item(string name, double weight, double value, string image)
    {
        this.name = name;
        this.weight = weight;
        this.value = value;
        this.image = image;
    }

    public virtual string GetInformationString()
    {
        string temp =
            "Name: " + name + "\r\n" +
            "Weight: " + weight + "\r\n" +
            "Value: " + Math.Round(value) + " gold\r\n";
        return temp;
    }
}

public class Weapon : Item
{
    public string baseDamage { get; set; }

    public Weapon() : base("", 0, 0, "")
    {
        baseDamage = "";
    }

    public Weapon(string name, double weight, double value, string image, string baseDamage)
        : base(name, weight, value, image)
    {
        this.baseDamage = baseDamage;
    }

    public override string GetInformationString()
    {
        return base.GetInformationString() + "Base Damage: " + baseDamage;
    }
}

public class Armor : Item
{
    public string baseArmor { get; set; }

    public Armor() : base("", 0, 0, "")
    {
        baseArmor = "";
    }
}

```



```

    }

    public Armor(string name, double weight, double value, string image, string baseArmor)
        : base(name, weight, value, image)
    {
        this.baseArmor = baseArmor;
    }

    public override string GetInformationString()
    {
        return base.GetInformationString() + "Base Armor Class: " + baseArmor;
    }
}
#endregion
}

```

Serialization.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Xml.Serialization;

namespace player_inventory
{
    public class Serialization
    {
        public static void Serialize<T>(T value, string path)
        {
            XmlSerializer serializer = new XmlSerializer(typeof(T));
            TextWriter writer = new StreamWriter(path);
            serializer.Serialize(writer, value);
            writer.Close();
        }

        public static T Deserialize<T>(string path)
        {
            XmlSerializer deserializer = new XmlSerializer(typeof(T));
            TextReader reader = new StreamReader(path);
            object obj = deserializer.Deserialize(reader);
            reader.Close();
            return (T)obj;
        }
    }
}

```



## Visual Basic Source Code Example

Form1.cs

Imports System

Imports System.Collections.Generic

Imports System.ComponentModel

Imports System.Data

Imports System.Drawing

Imports System.Linq

Imports System.Text

Imports System.Threading.Tasks

Imports System.Windows.Forms

Imports System.Xml

Imports System.Xml.Serialization

Namespace player\_inventory

Public Partial Class Form1

Inherits Form

Private allItems As List(Of Item) = New List(Of Item)()

Private player As Character = New Character()

Private shop As Character = New Character()

Public Sub New()

InitializeComponent()

Try

allItems = Serialization.Deserialize(Of List(Of Item))("all\_items.xml")

Catch

GenerateAllItemsFile()

End Try

NewPlayer()

NewShop()

End Sub

Private Sub GenerateAllItemsFile()

allItems.Add(New Weapon("Greatsword", 15.0, 100, "../../../images/greatsword.png", "2d6"))

allItems.Add(New Item("Potion", 0.5, 10, "../../../images/potion.png"))

allItems.Add(New Item("Bag of Holding", 1.0, 200, "../../../images/bag\_of\_holding.png"))

allItems.Add(New Weapon("Battleaxe", 7.0, 80, "../../../images/battleaxe.png", "1d8"))

allItems.Add(New Armor("Blue Dragon Armor", 40.0, 300, "../../../images/blue\_dragon\_armor.png", "8"))

Serialization.Serialize(allItems, "all\_items.xml")

End Sub

Private Sub NewPlayer()

player = CreateRandomCharacter(4, 200, 200)

UpdatePlayerGui()

End Sub

Private Sub NewShop()

shop = CreateRandomCharacter(10, 1000, 200)

UpdateVendorGui()

End Sub

Private Function CreateRandomCharacter(ByVal numItems As Integer, ByVal baseGold As Double, ByVal randomExtraGold As Double) As Character





## ANSWER KEY

Page 17 of 21

```

Dim temp As Character = New Character()
Dim rng As Random = New Random()

For i As Integer = 0 To numItems - 1
    Dim randomIndex As Integer = CInt(Math.Round(rng.NextDouble() * (allItems.Count - 1)))
    temp.items.Add(allItems(randomIndex))
Next

temp.gold = rng.NextDouble() * randomExtraGold + baseGold
Return temp
End Function

Private Sub UpdateVendorGui()
    label_vendorGold.Text = Math.Round(shop.gold).ToString() & " gold"
    shopList.DataSource = Nothing
    shopList.DataSource = shop.items
    shopList.DisplayMember = "name"

    If (shopList.SelectedIndex = -1) AndAlso (shop.items.Count <> 0) Then
        shopList.SelectedIndex = 0
    End If
End Sub

Private Sub UpdatePlayerGui()
    label_playerGold.Text = Math.Round(player.gold).ToString() & " gold"
    playerList.DataSource = Nothing
    playerList.DataSource = player.items
    playerList.DisplayMember = "name"

    If (playerList.SelectedIndex = -1) AndAlso (player.items.Count <> 0) Then
        playerList.SelectedIndex = 0
    End If
End Sub

Private Sub SetItemInfo(ByVal item As Item)
    itemInfoPictureBox.Image = Image.FromFile(item.image)
    itemInfoLabel.Text = item.GetInformationString()
End Sub

Private Sub shopList_SelectedIndexChanged(ByVal sender As Object, ByVal e As EventArgs)
    If shopList.SelectedItem IsNot Nothing Then
        SetItemInfo(CType(shopList.SelectedItem, Item))
    End If
End Sub

Private Sub playerList_SelectedIndexChanged(ByVal sender As Object, ByVal e As EventArgs)
    If playerList.SelectedItem IsNot Nothing Then
        SetItemInfo(CType(playerList.SelectedItem, Item))
    End If
End Sub

Private Sub refreshShopButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    NewShop()
End Sub

Private Sub exitButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim r1 As DialogResult = MessageBox.Show("Are you sure you wish to exit?", "Confirm",
    MessageBoxButtons.YesNo)

```



```
If r1 = DialogResult.Yes Then
    Application.[Exit]()
End If
End Sub

Private Sub sellButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    If playerList.SelectedIndex <> -1 Then

        If shop.BuyItem(player.items(playerList.SelectedIndex)) Then
            player.SellItem(playerList.SelectedIndex)
        End If

        UpdatePlayerGui()
        UpdateVendorGui()
    End If
End Sub

Private Sub buyButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    If shopList.SelectedIndex <> -1 Then

        If player.BuyItem(shop.items(shopList.SelectedIndex)) Then
            shop.SellItem(shopList.SelectedIndex)
        End If

        UpdatePlayerGui()
        UpdateVendorGui()
    End If
End Sub

Private Sub autoQuestButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim rng As Random = New Random()
    player.gold += rng.NextDouble() * 20 + 20
    UpdatePlayerGui()
End Sub

Private Sub loadButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim openFileDialog1 As OpenFileDialog = New OpenFileDialog()
    openFileDialog1.InitialDirectory = Environment.CurrentDirectory

    If openFileDialog1.ShowDialog() = DialogResult.OK Then

        Try
            player = Serialization.Deserialize(Of Character)(openFileDialog1.FileName)
        Catch
            MessageBox.Show("Invalid player file selected!", "Warning")
        End Try
    End If

    UpdatePlayerGui()
End Sub

Private Sub saveButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim saveFileDialog1 As SaveFileDialog = New SaveFileDialog()
    saveFileDialog1.InitialDirectory = Environment.CurrentDirectory

    If saveFileDialog1.ShowDialog() = DialogResult.OK Then
```



## ANSWER KEY

Page 19 of 21

```

        Try
            Serialization.Serialize(player, saveFileDialog1.FileName)
        Catch
            MessageBox.Show("Invalid player file selected!", "Warning")
        End Try
    End If
End Sub
End Class

Public Class Character
    Public Property gold As Double
    Public Property items As List(Of Item)

    Public Sub New()
        items = New List(Of Item)()
        gold = 0
    End Sub

    Public Function BuyItem(ByVal item As Item) As Boolean
        Dim status As Boolean = True

        If gold < item.value Then
            MessageBox.Show("Insufficient gold!", "Warning")
            status = False
        Else
            gold -= item.value
            items.Add(item)
        End If

        Return status
    End Function

    Public Function SellItem(ByVal itemIndex As Integer) As Boolean
        gold += items(itemIndex).value
        items.RemoveAt(itemIndex)
        Return True
    End Function
End Class

<XmlInclude(GetType(Weapon))>
<XmlInclude(GetType(Armor))>
Public Class Item
    Public Property name As String
    Public Property weight As Double
    Public Property value As Double
    Public Property image As String

    Public Sub New()
    End Sub

    Public Sub New(ByVal name As String, ByVal weight As Double, ByVal value As Double, ByVal image As
String)
        Me.name = name
        Me.weight = weight
        Me.value = value
        Me.image = image
    End Sub

```



**ANSWER KEY**

**Page 20 of 21**

```

    Public Overridable Function GetInformationString() As String
        Dim temp As String = "Name: " & name & vbCrLf & "Weight: " & weight & vbCrLf & "Value: " &
Math.Round(value) & " gold" & vbCrLf
        Return temp
    End Function
End Class

Public Class Weapon
    Inherits Item

    Public Property baseDamage As String

    Public Sub New()
        MyBase.New("", 0, 0, "")
        baseDamage = ""
    End Sub

    Public Sub New(ByVal name As String, ByVal weight As Double, ByVal value As Double, ByVal image As
String, ByVal baseDamage As String)
        MyBase.New(name, weight, value, image)
        Me.baseDamage = baseDamage
    End Sub

    Public Overrides Function GetInformationString() As String
        Return MyBase.GetInformationString() & "Base Damage: " & baseDamage
    End Function
End Class

Public Class Armor
    Inherits Item

    Public Property baseArmor As String

    Public Sub New()
        MyBase.New("", 0, 0, "")
        baseArmor = ""
    End Sub

    Public Sub New(ByVal name As String, ByVal weight As Double, ByVal value As Double, ByVal image As
String, ByVal baseArmor As String)
        MyBase.New(name, weight, value, image)
        Me.baseArmor = baseArmor
    End Sub

    Public Overrides Function GetInformationString() As String
        Return MyBase.GetInformationString() & "Base Armor Class: " & baseArmor
    End Function
End Class
End Namespace

Serialization.vb
Imports System
Imports System.Collections.Generic
Imports System.IO
Imports System.Linq
Imports System.Text
Imports System.Threading.Tasks
Imports System.Xml.Serialization

```



```
Namespace player_inventory
    Public Class Serialization
        Public Shared Sub Serialize(Of T)(ByVal value As T, ByVal path As String)
            Dim serializer As XmlSerializer = New XmlSerializer(GetType(T))
            Dim writer As TextWriter = New StreamWriter(path)
            serializer.Serialize(writer, value)
            writer.Close()
        End Sub

        Public Shared Function Deserialize(Of T)(ByVal path As String) As T
            Dim deserializer As XmlSerializer = New XmlSerializer(GetType(T))
            Dim reader As TextReader = New StreamReader(path)
            Dim obj As Object = deserializer.Deserialize(reader)
            reader.Close()
            Return CType(obj, T)
        End Function
    End Class
End Namespace
```